



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Задание по практической работе

по дисциплине «Моделирование программных систем»

Выполнили:

Студенты группы ИКБО-66-23

Смирнов А.Ю.

Хомченко В.А.

Ян Хуацзи

Проверил:

Образцов В.М.

2025 г.

Оглавление

Задание.....	3
Часть 1. Создание диаграммы процесса. Изменение свойств блоков модели, её настройка и запуск.....	5
Часть 2. Создание анимации модели.....	6
Часть 3. Сбор статистики использования ресурсов.....	7
Часть 4. Уточнение модели согласно ёмкости входного буфера	9
Часть 5. Сбор статистики по показателям обработки запросов	10
Часть 6. Добавление параметров и элементов управления.....	14
Часть 7. Добавление гистограмм.....	17
Часть 8. Изменение времени обработки запросов сервером	19
Часть 9. Интерпретация результатов моделирования	21
Вывод	22

Задание

Цель работы: получение экспериментальной модели обработки запросов сервером.

Постановка задачи:

Построить модель работы сервера используя сети Петри. Использовать в качестве инструмента имитационного моделирования – Anylogic 8 PLE (бесплатная версия).

Модель обработки запросов сервером

Сервер обрабатывает запросы, поступающие с автоматизированных рабочих мест с интервалами, распределенными по показательному закону со средним значением 2 мин. Время обработки сервером одного запроса распределено по экспоненциальному закону со средним значением 3 мин. Сервер имеет входной буфер ёмкостью 5 запросов.

Сервер представляет собой однофазную систему массового обслуживания разомкнутого типа с ограниченной входной ёмкостью, то есть с отказами, и абсолютной надёжностью.

Часть 1. Создание диаграммы процесса. Изменение свойств блоков модели, её настройка и запуск

1. Выполните Файл/Создать/Модель на панели инструментов. Появится диалоговое окно Новая модель. Задайте имя новой модели. В поле Имя модели введите Server.

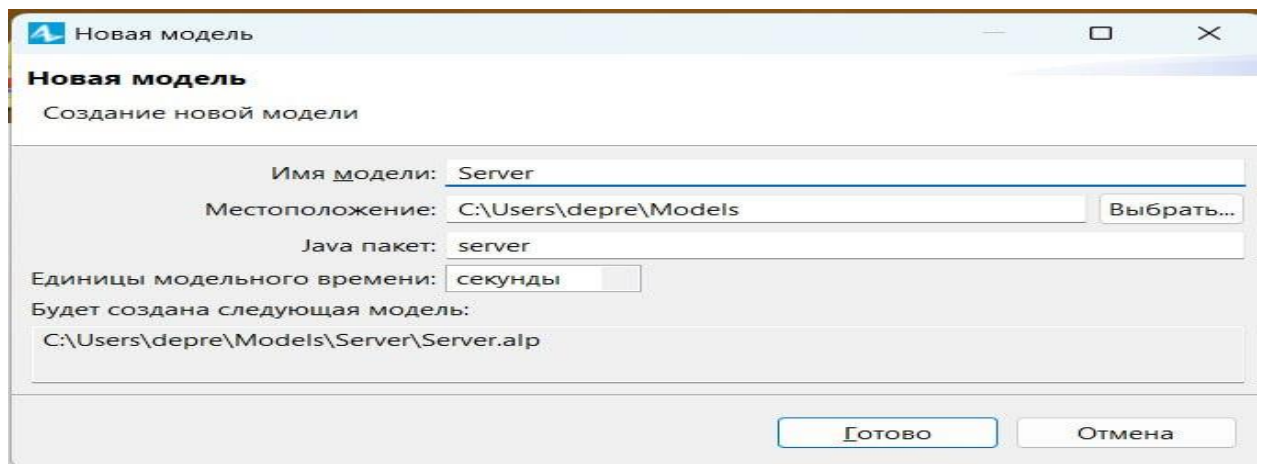


Рисунок 1 – Создание новой модели

2. Создайте диаграмму процесса. Для этого в Палитре выделите Библиотеку моделирования процессов. Из неё перетащите объекты на диаграмму и соедините.

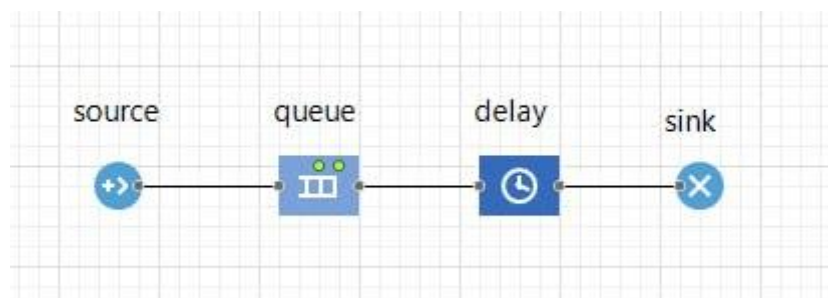


Рисунок 2 – Создание диаграммы процесса

3. Измените свойства объектов и запустите модель.



Рисунок 3 – Запуск модели

Часть 2. Создание анимации модели

4. Нарисуйте прямоугольный узел, который будет обозначать на анимации сервер. Нарисуйте путь, который будет обозначать на анимации очередь к серверу.

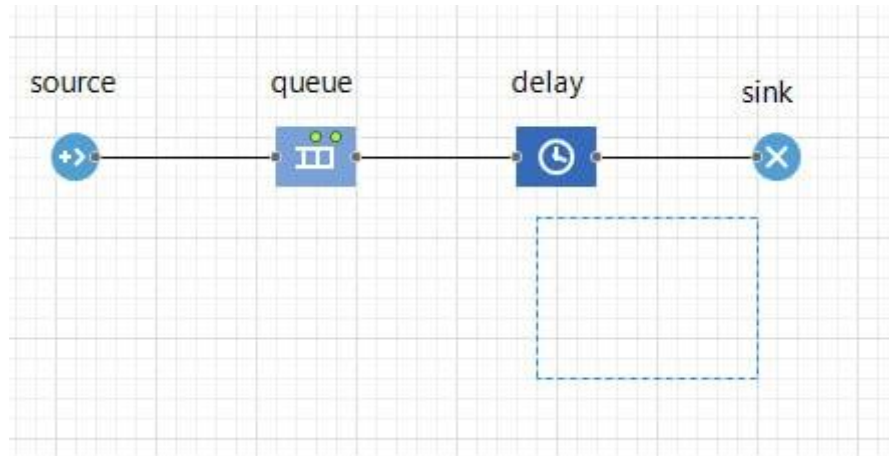


Рисунок 4 – Добавление прямоугольного узла

5. Теперь мы должны задать созданные анимационные объекты в качестве анимационных фигур объектов диаграммы нашего процесса. Задайте путь в качестве фигуры анимации очереди.

queue - Queue

Имя: ☒ Отобража

☐ Исключить

Вместимость:

Максимальная вместимость:

Место агентов:  

Рисунок 5 – Установка места агентов у очереди

6. Задайте прямоугольный узел в качестве фигуры анимации сервера.

delay - Delay

Имя: ☒ Отображать им

☐ Исключить

Тип задержки: ☒ Определенное время ☐ До вызова функции stopDelay()

Время задержки:

Вместимость:

Максимальная вместимость:

Место агентов:  

Рисунок 6 – Установка места агентов у задержки

7. Запустите модель. Вы увидите, что у модели теперь есть простейшая анимация — сервер и очередь запросов к нему. Цвет фигуры сервера будет меняться в зависимости от того, обрабатывается ли запрос в данный момент времени или нет.

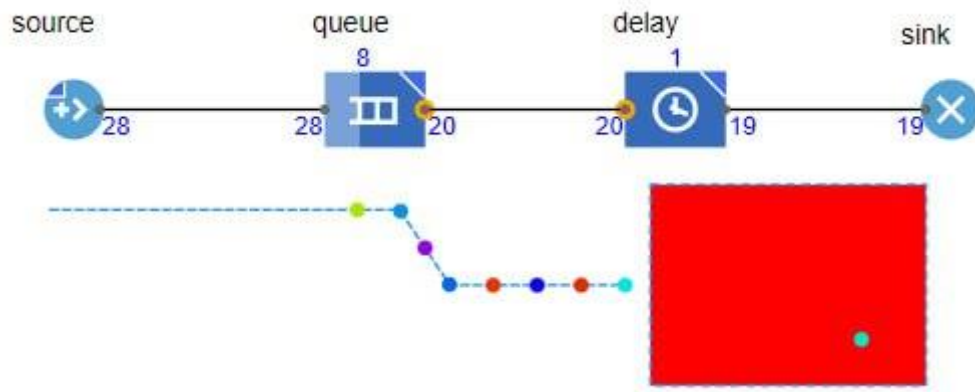


Рисунок 7 – Запуск модели

Часть 3. Сбор статистики использования ресурсов

8. Добавьте диаграмму для отображения среднего коэффициента использования сервера.

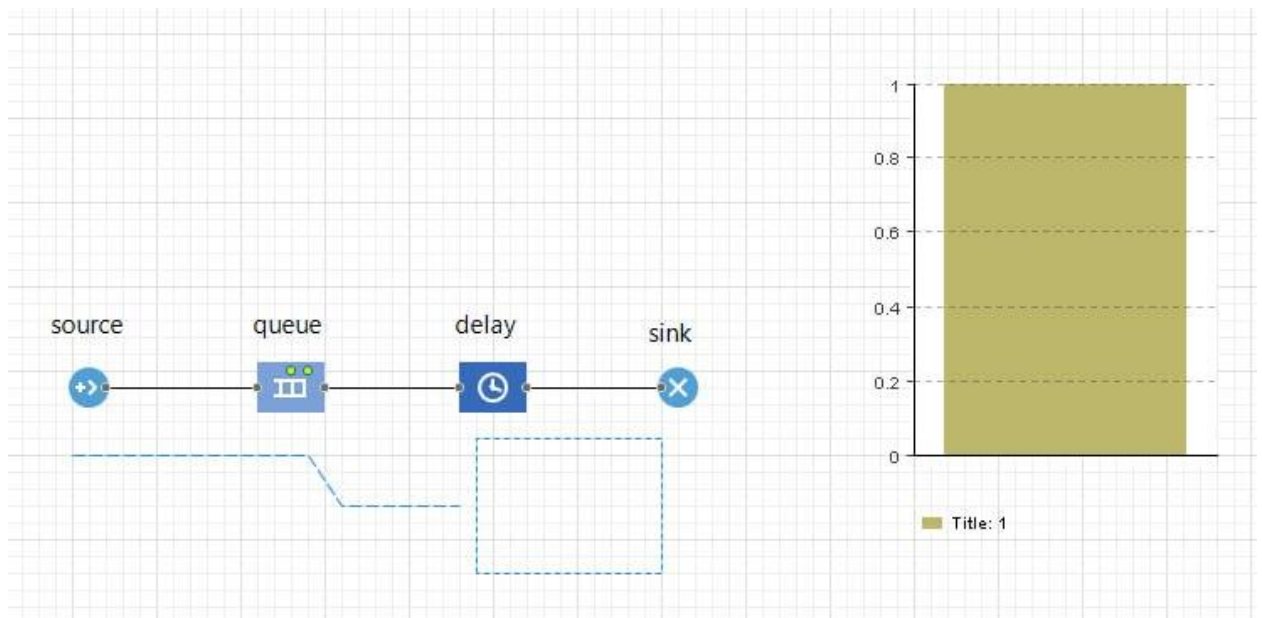


Рисунок 8 – Добавление диаграммы

9. Аналогичным образом добавьте еще одну столбиковую диаграмму для отображения средней длины очереди.

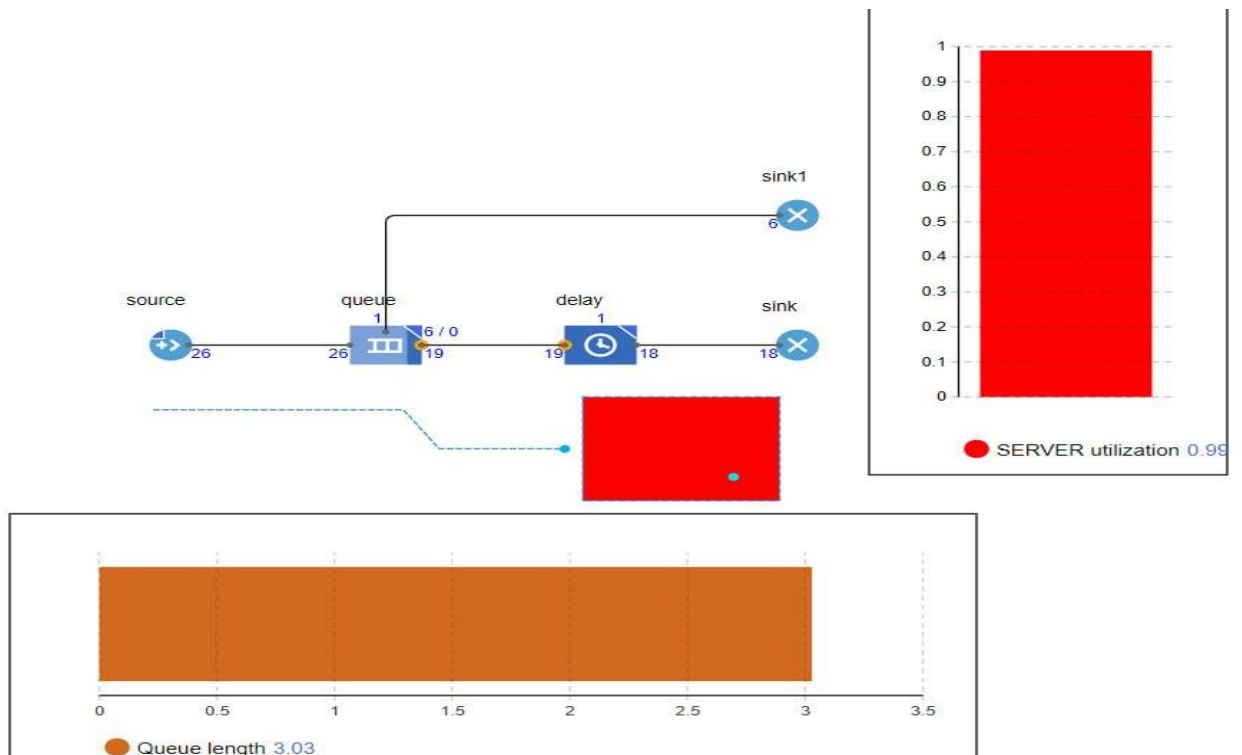


Рисунок 9 – Добавление диаграммы

10. Запустите модель с двумя столбиковыми диаграммами, установив модельное время 3600 единиц, и понаблюдайте за её работой.

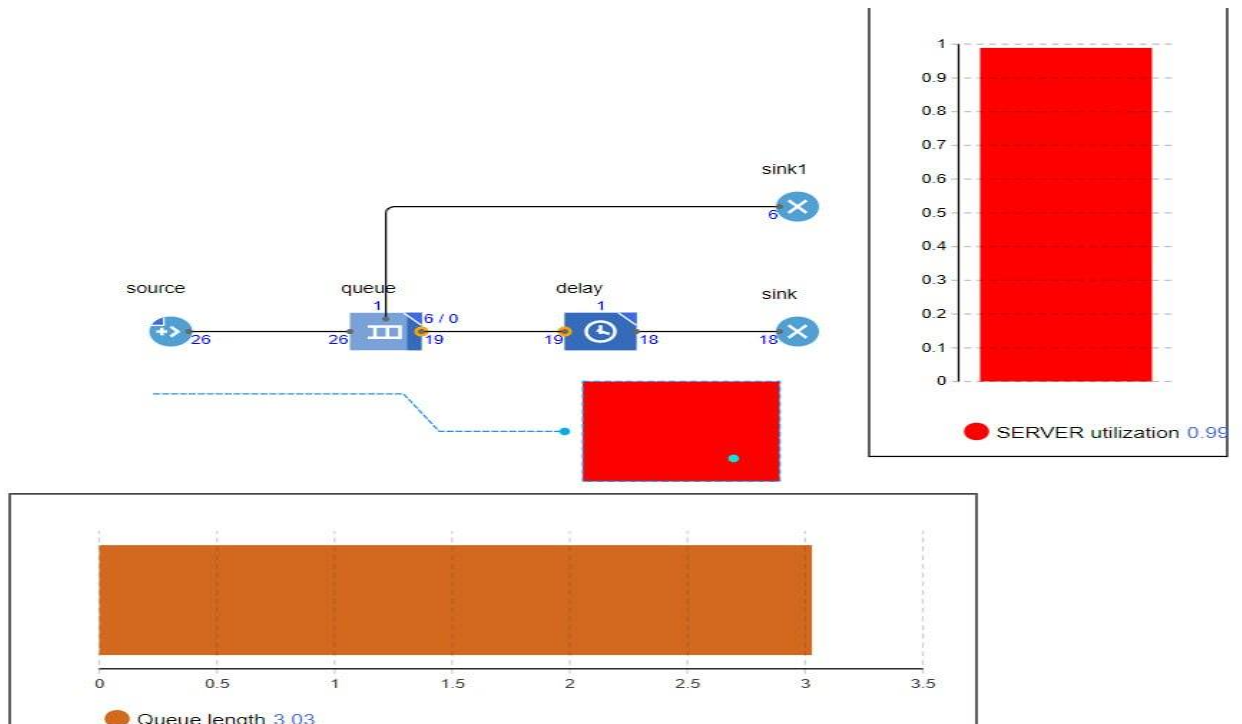


Рисунок 10 – Запуск модели

Часть 4. Уточнение модели согласно ёмкости входного буфера

11. Для уничтожения потерянных запросов вследствие полного заполнения накопителя нужно добавить второй объект sink. Соедините порт outPreempted объекта queue с входным портом InPort блока sink1.

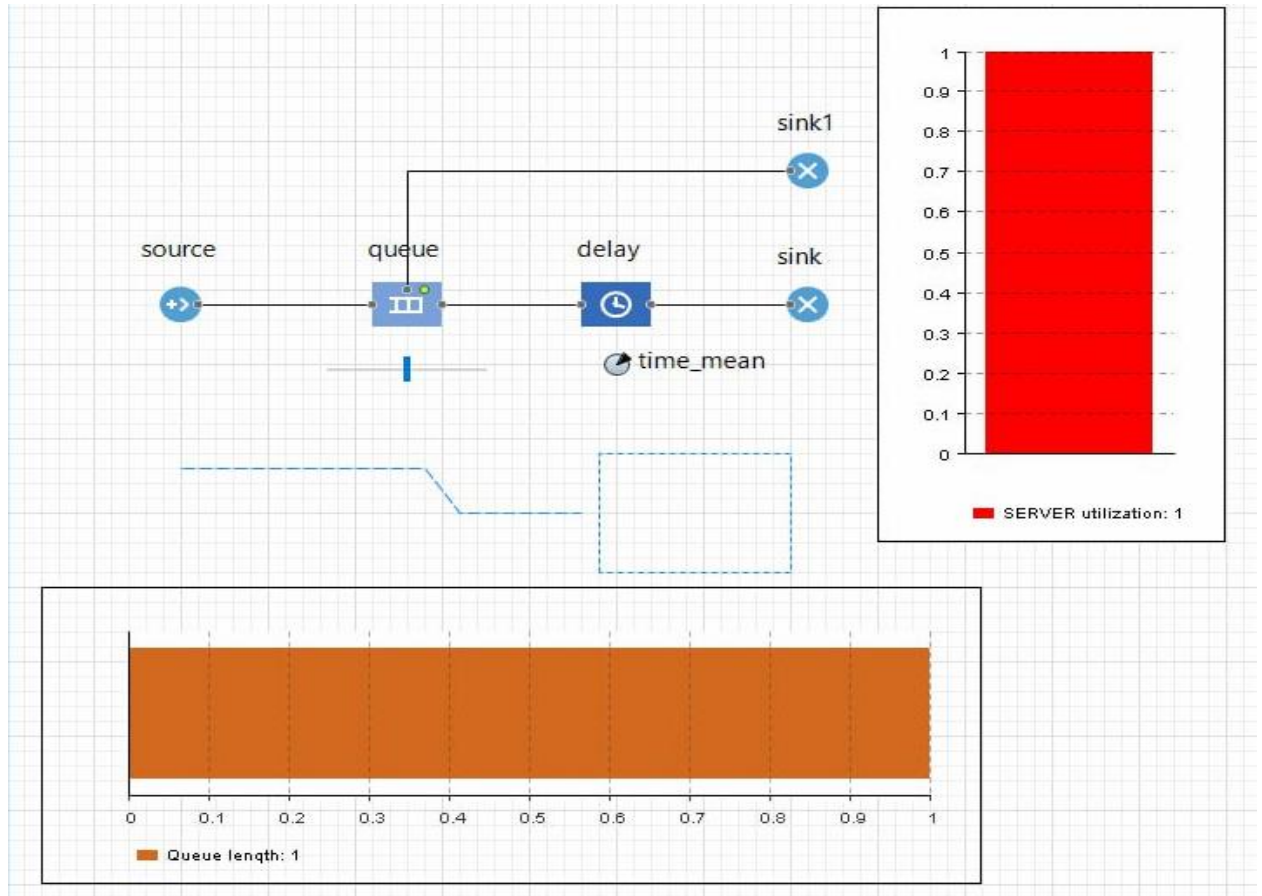


Рисунок 11 – Добавление вытеснения заявок

Часть 5. Сбор статистики по показателям обработки запросов

12. Для включения в запросы дополнительных полей необходимо создать нестандартный тип заявки. Это возможно двумя способами. Создадим первым способом тип заявок Inquiry. Создадим первым способом тип заявок Inquiry.

```
3  */
4 public class Inquiry extends Entity implements Serializable {
5
6     double time_vxod;
7
8     double time_vixod;
9
10    int col_vxod;
11
12    int col_vixod;
13
14    /**
15     * Конструктор по умолчанию
16     */
17    public Inquiry() {
18    }
19
20    /**
21     * Конструктор, инициализирующий поля
22     */
23    public Inquiry(double time_vxod, double time_vixod, int col_vxod, int col_vixod) {
24        this.time_vxod = time_vxod;
25        this.time_vixod = time_vixod;
26        this.col_vxod = col_vxod;
27        this.col_vixod = col_vixod;
28    }
29
30    @Override
31    public String toString() {
32        return
33            "time_vxod = " + time_vxod + " " +
34            "time_vixod = " + time_vixod + " " +
35            "col_vxod = " + col_vxod + " " +
36            "col_vixod = " + col_vixod + " ";
37    }
38
39    /**
40     * Это число используется при сохранении состояния модели<br>
41     * Его рекомендуется изменить в случае изменения класса
42     */
43    private static final long serialVersionUID = 1L;
44
45 }
```

Рисунок 12 – Создание нового типа агента через создание нового Java класса

13. Появится окно с автоматически созданными параметрами нестандартного типа заявок Inquiry.

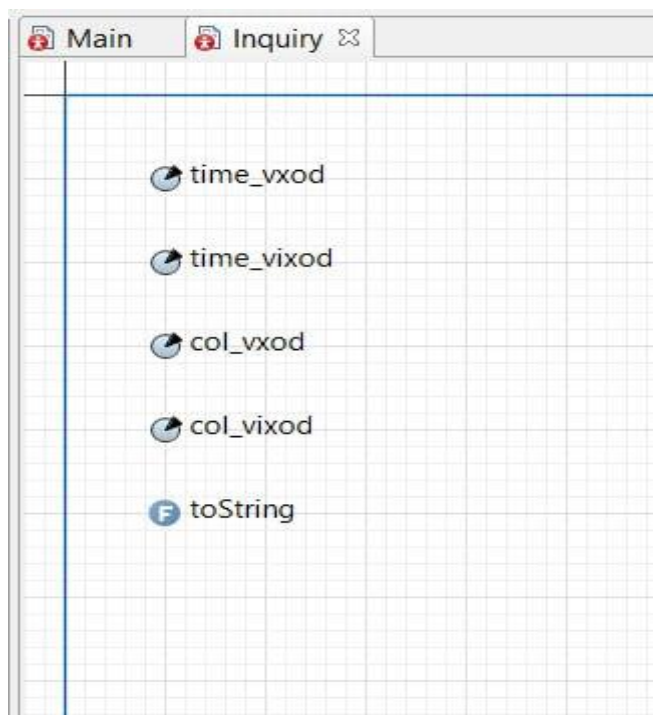


Рисунок 13 – Новый созданный класс

14. Перейдём к созданию непосредственно нестандартного типа заявки вторым способом. Выберите анимацию агента: установите 2D и выберите из выпадающего списка, например, Сообщение.

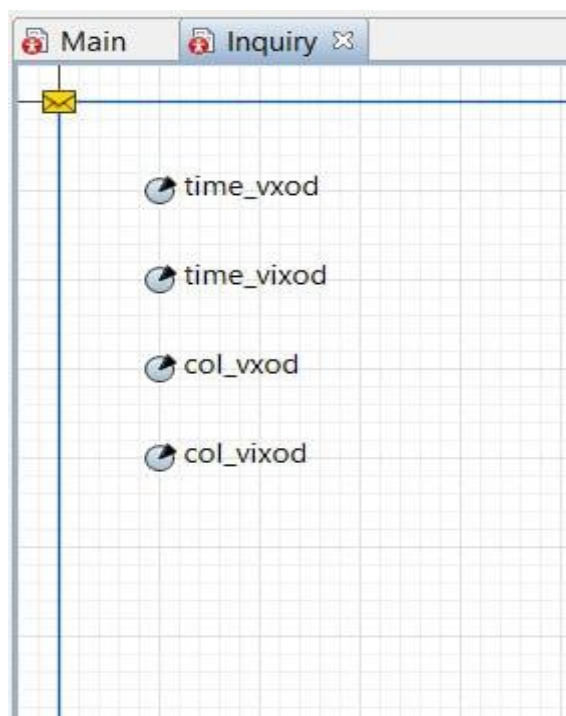


Рисунок 14 – Создание нового класса

15. Для сбора статистических данных о времени обработки запросов сервером необходимо добавить элемент статистики. Этот элемент будет запоминать значения времен для каждого запроса. На основе этого он предоставит пользователю стандартную статистическую информацию (среднее, минимальное, максимальное из измеренных значений, среднеквадратичное отклонение и т.д.).

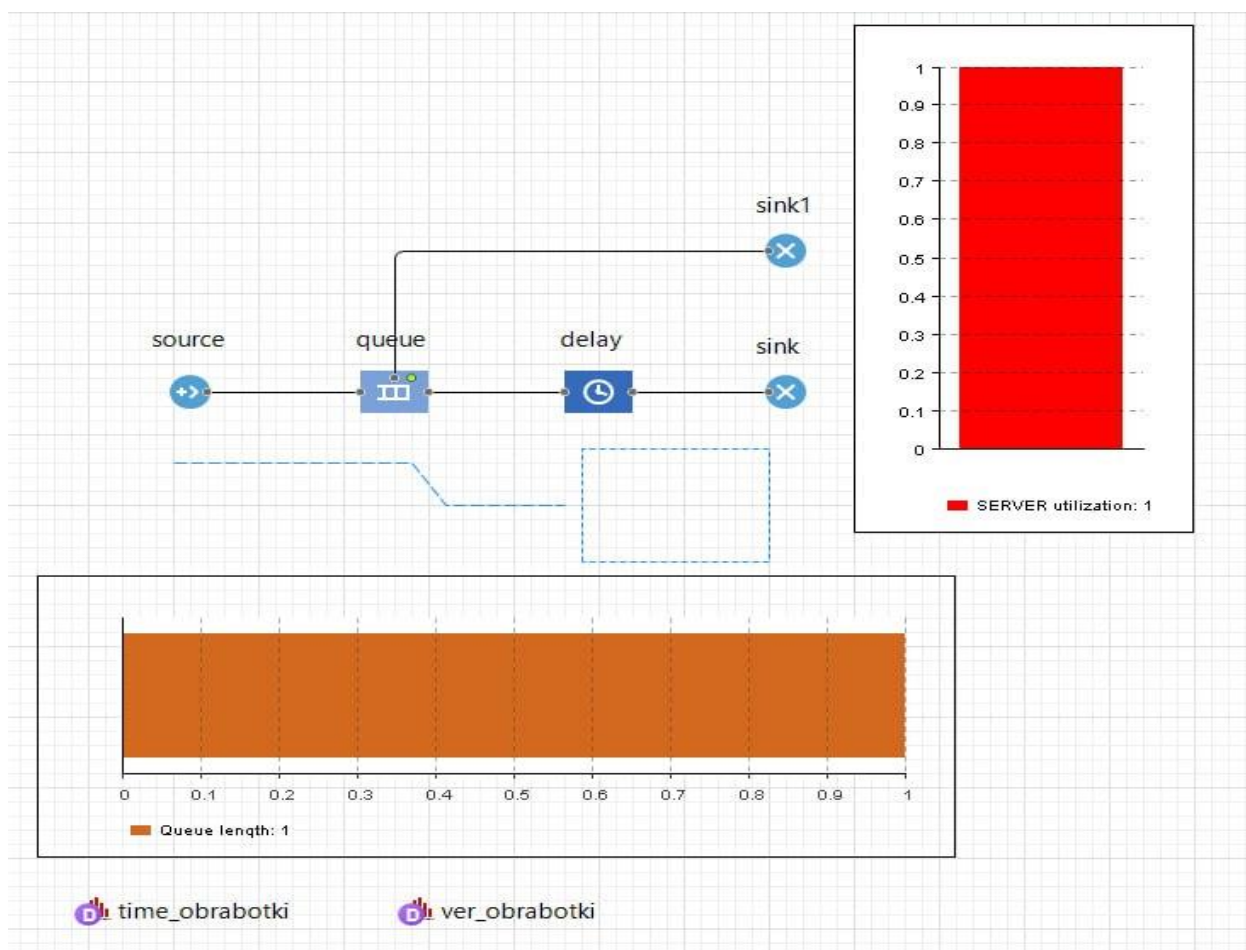


Рисунок 15 – Добавление элементов статистики

16. Измените код при входе элемента sink.

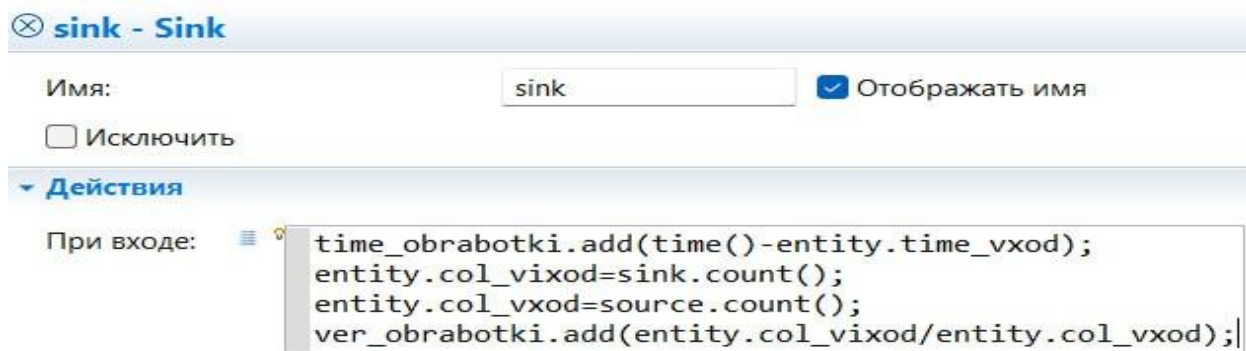


Рисунок 16 – Измененный код объекта sink

17. Обратите внимание, что вам не пришлось использовать поле `time_vxod`, так как вместо него была использована функция `time()`, возвращающая, как вам уже известно, текущее значение модельного времени. Удалите поле `time_vxod`.

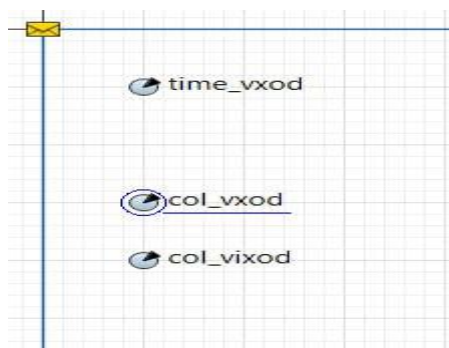


Рисунок 17 – Удалено поле `time_vxod`

18. Итак, все условия постановки задачи выполнены. Чтобы наблюдать за работой модели, установите, что время остановки модели не задано. Запустите модель.

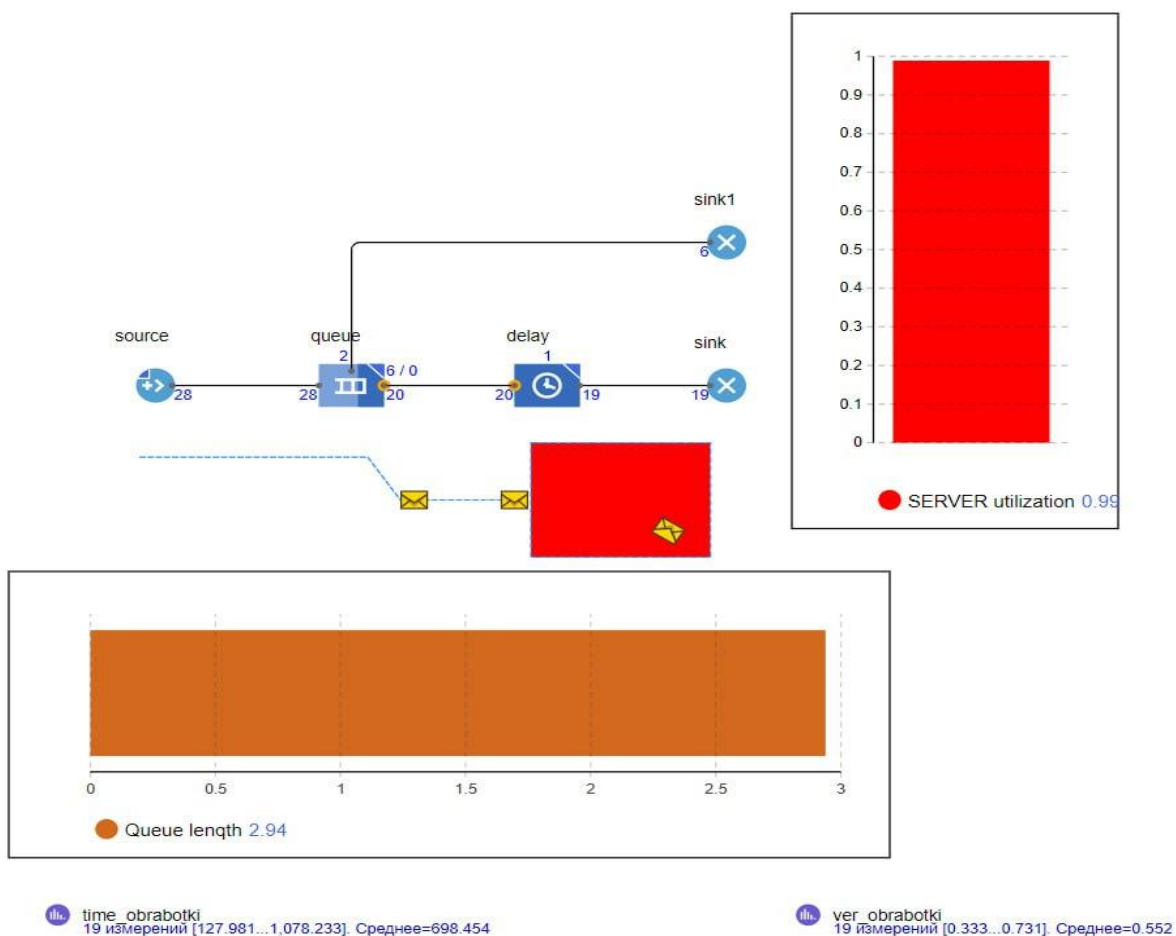


Рисунок 18 – Запуск модели

Часть 6. Добавление параметров и элементов управления

19. Активный объект может иметь параметры. Параметры обычно используются для задания статических характеристик объекта. Но значения параметров при необходимости можно изменять во время работы модели. Для этого нужно написать код обработчика события, то есть действий, которые должны выполняться при изменении значения параметра. Создайте параметр `time_mean` объекта `delay`.

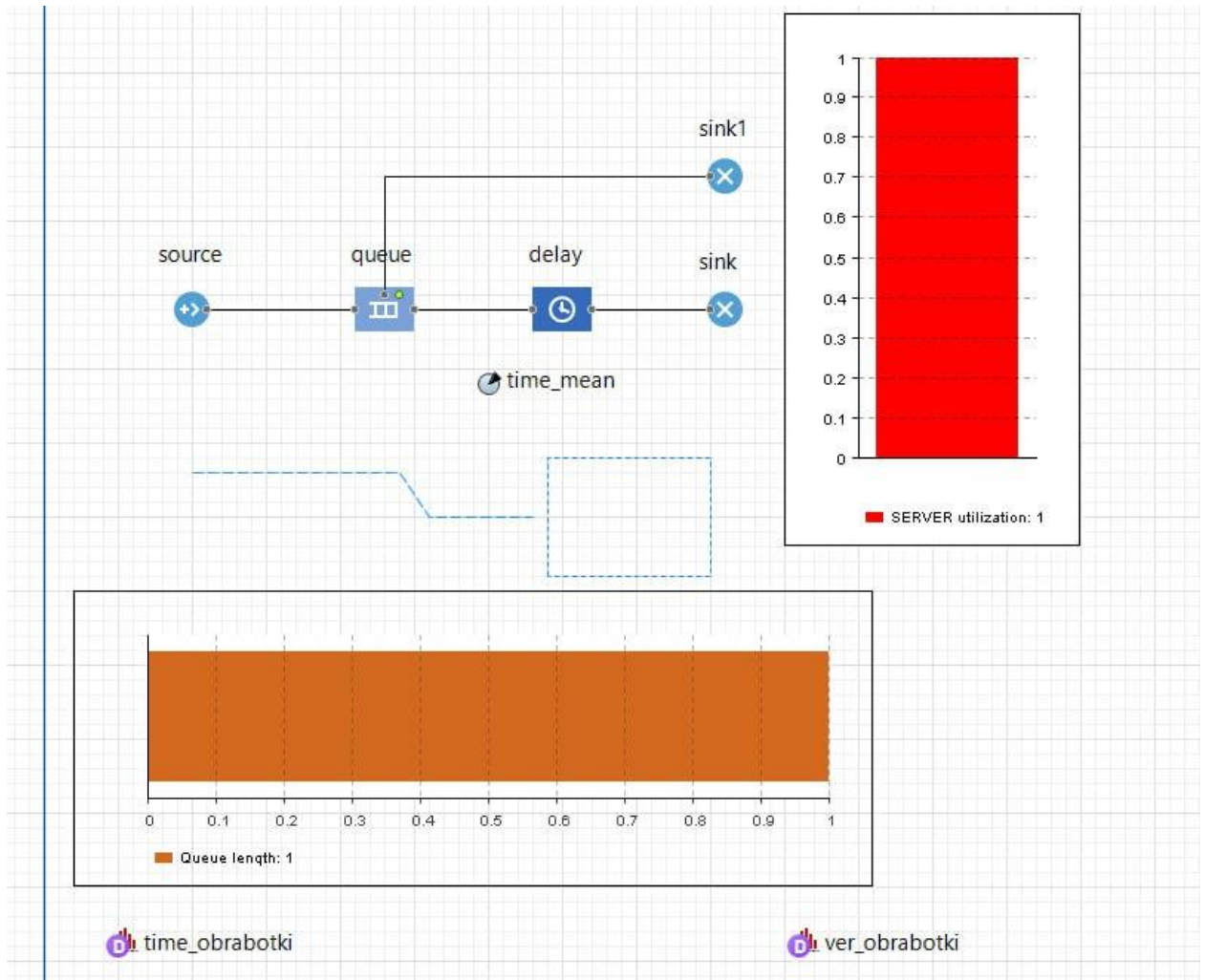


Рисунок 19 – Добавлен параметр `time_mean`

20. Пусть вы хотите изменять среднее время обработки запросов `time_mean` в ходе моделирования. Используйте для этого элемент управления — бегунок.

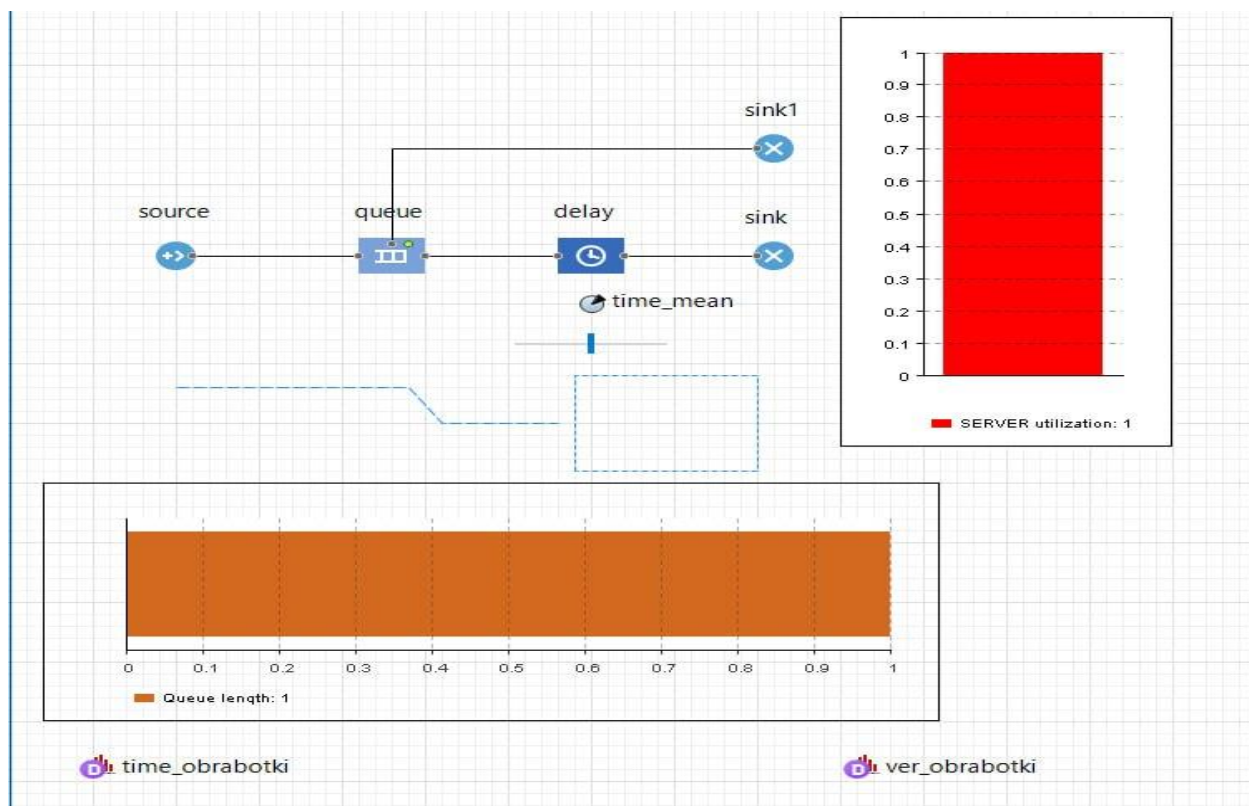


Рисунок 20 – Добавление ползунка

21. Пусть теперь вы хотите также изменять ёмкость буфера в ходе моделирования. Используйте для этого также бегунок.

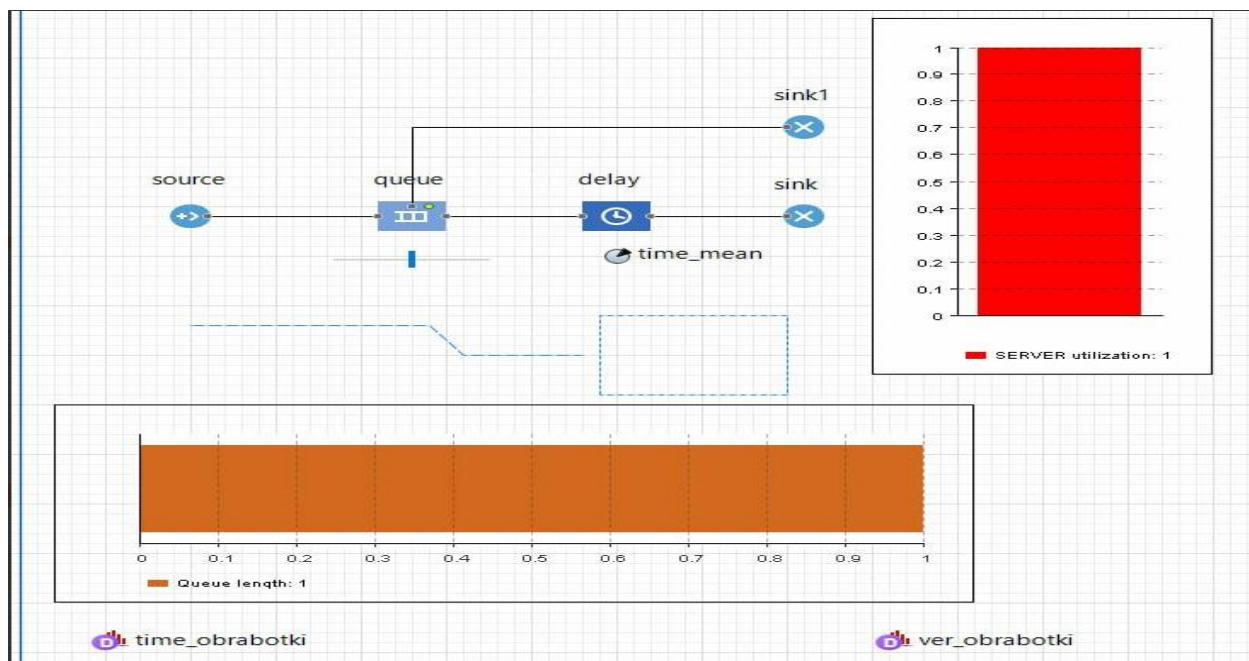


Рисунок 21 – Добавление ползунка

22. Существует и другой способ изменения свойств объектов во время выполнения модели: нужно щёлкнуть по элементу, войти в режим редактирования и ввести новое значение в одной из закладок всплывающего окна инспекта. Поэтому заранее не нужно продумывать, значения каких параметров планируется изменять, и не добавлять специальные элементы управления (например, бегунки).

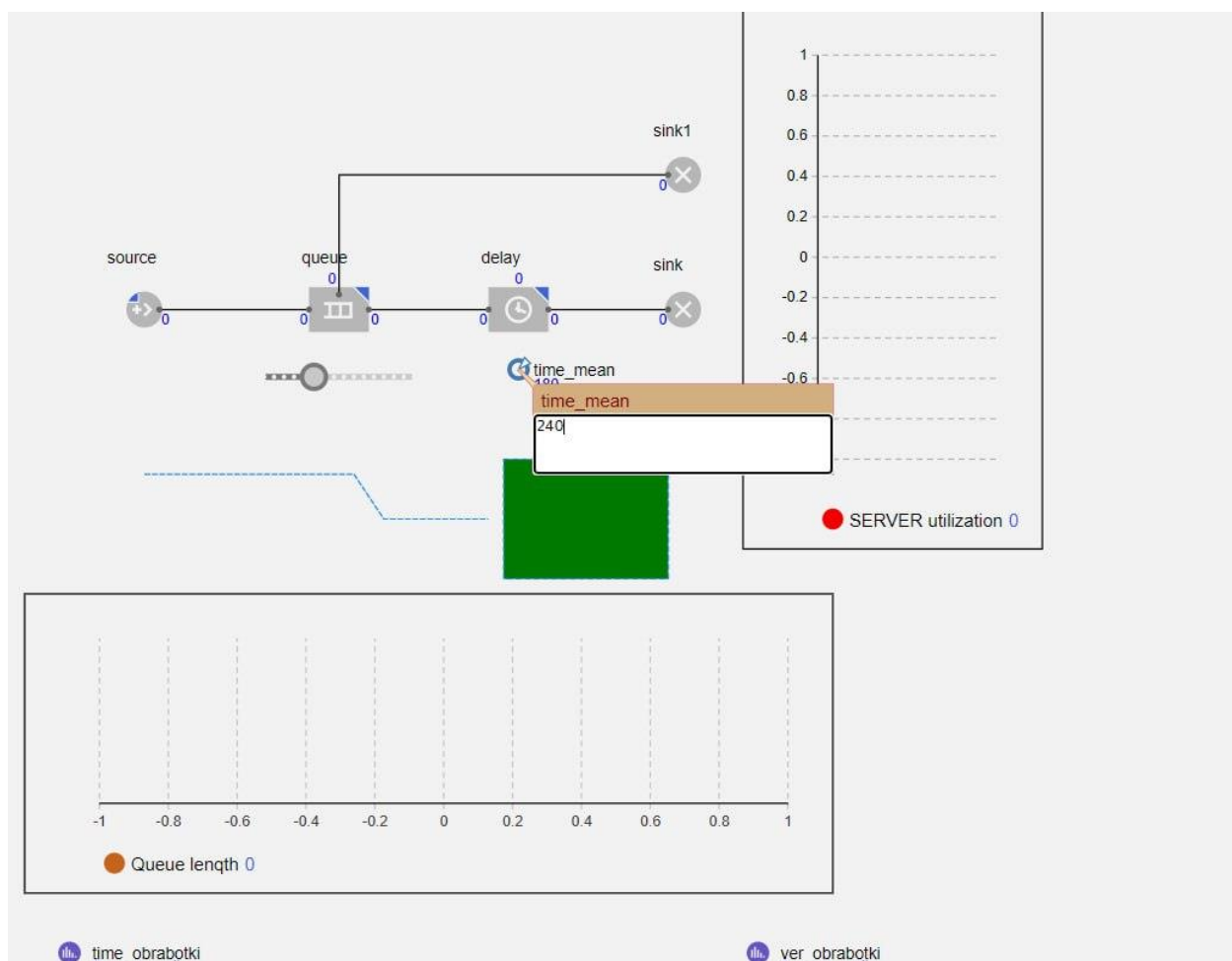


Рисунок 22 – Динамическое изменение параметра

Часть 7. Добавление гистограмм

23. Теперь добавим на диаграмму нашего потока гистограмму, которая будет отображать собранную временную статистику.

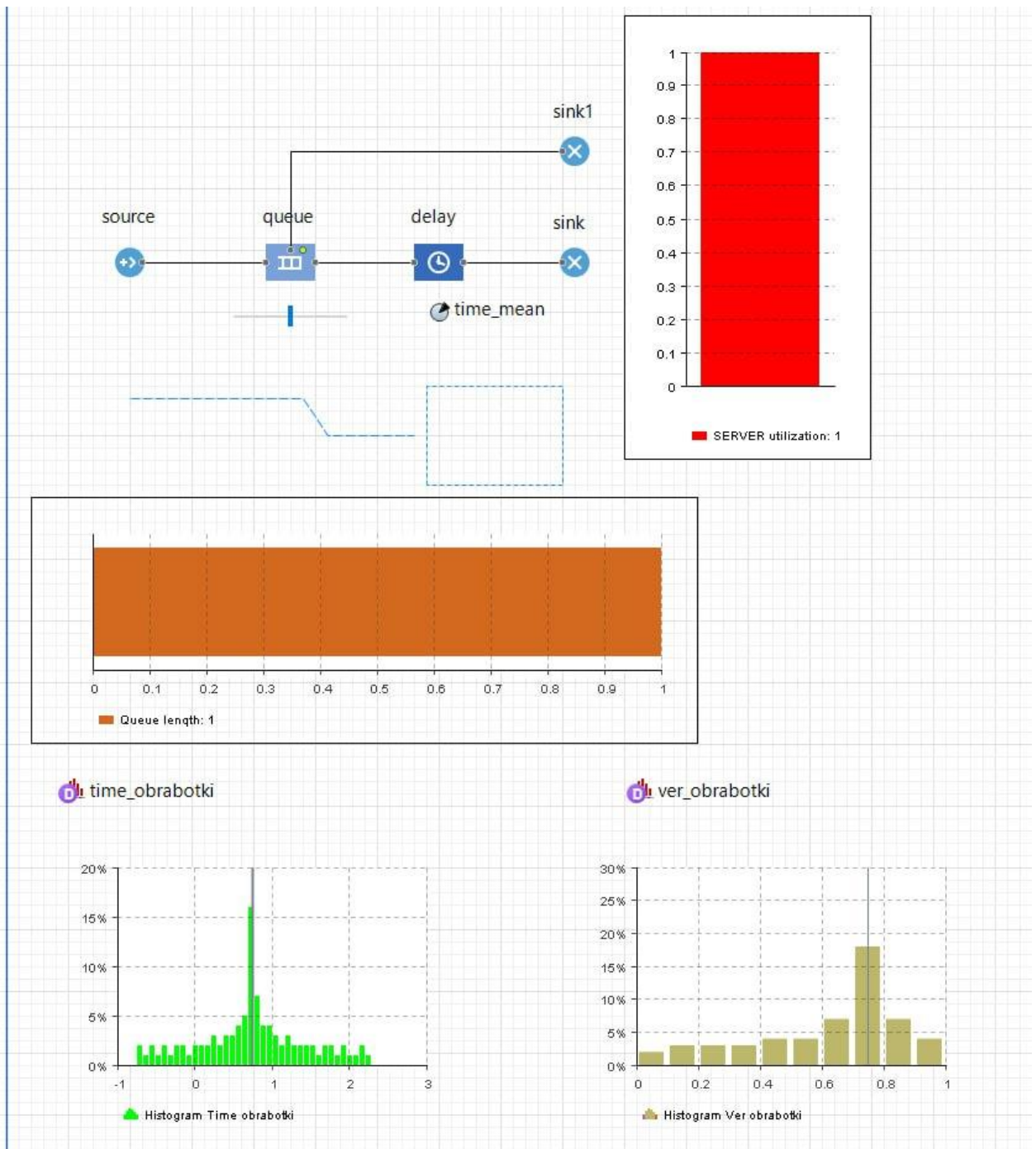


Рисунок 23 – Добавление гистограмм

24. Запустите модель.

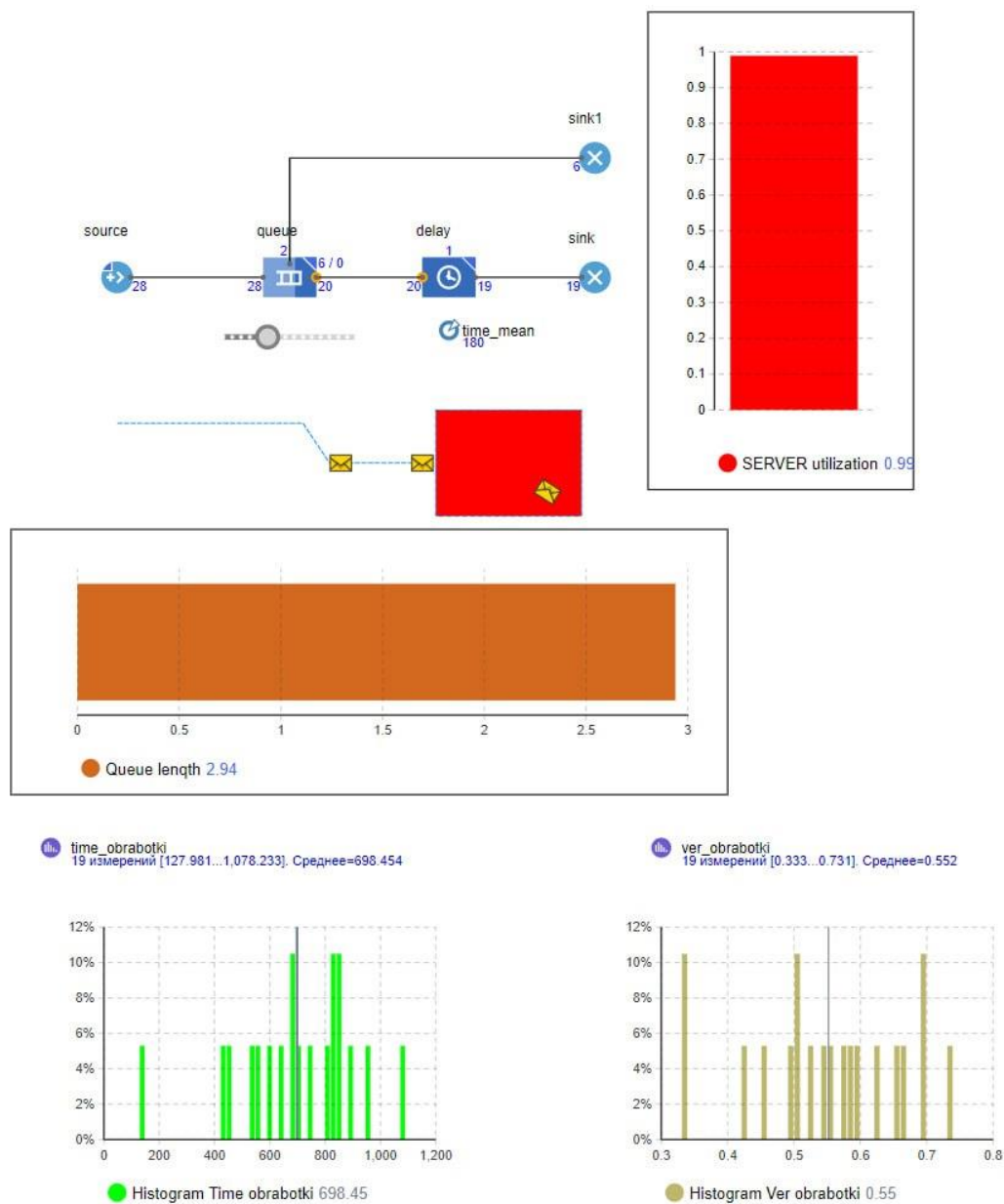


Рисунок 24 – Запуск модели

Часть 8. Изменение времени обработки запросов сервером

25. Внесите в модель изменения для аналогичного расчёта времени обработки запросов. Из палитры Основная перетащите три элемента Параметр на диаграмму класса Main.

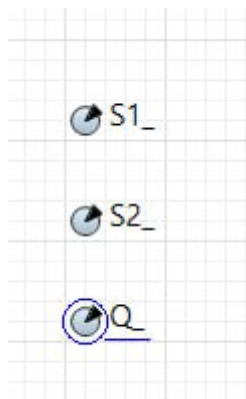


Рисунок 25 – Добавление параметров

26. Показатели моделируемой системы нужно определить в течение 3600 с, поэтому время моделирования в AnyLogic составит $3600 \cdot 9604 = 34574400$ единиц модельного времени.

Simulation - Простой эксперимент

Имя: ☐ Исключить

Агент верхнего уровня:

Максимальный размер памяти: Мб

☒ Пропустить экран эксперимента и запустить модель

▸ **Параметры**

▾ **Модельное время**

Режим выполнения: ☒ Виртуальное время (максимальная скорость)
☐ Реальное время со скоростью

Остановить:

Начальное время: Конечное время:

Начальная дата: Конечная дата:

Рисунок 26 – Изменение свойств эксперимента

27. Запустите модель и дождитесь окончания моделирования.

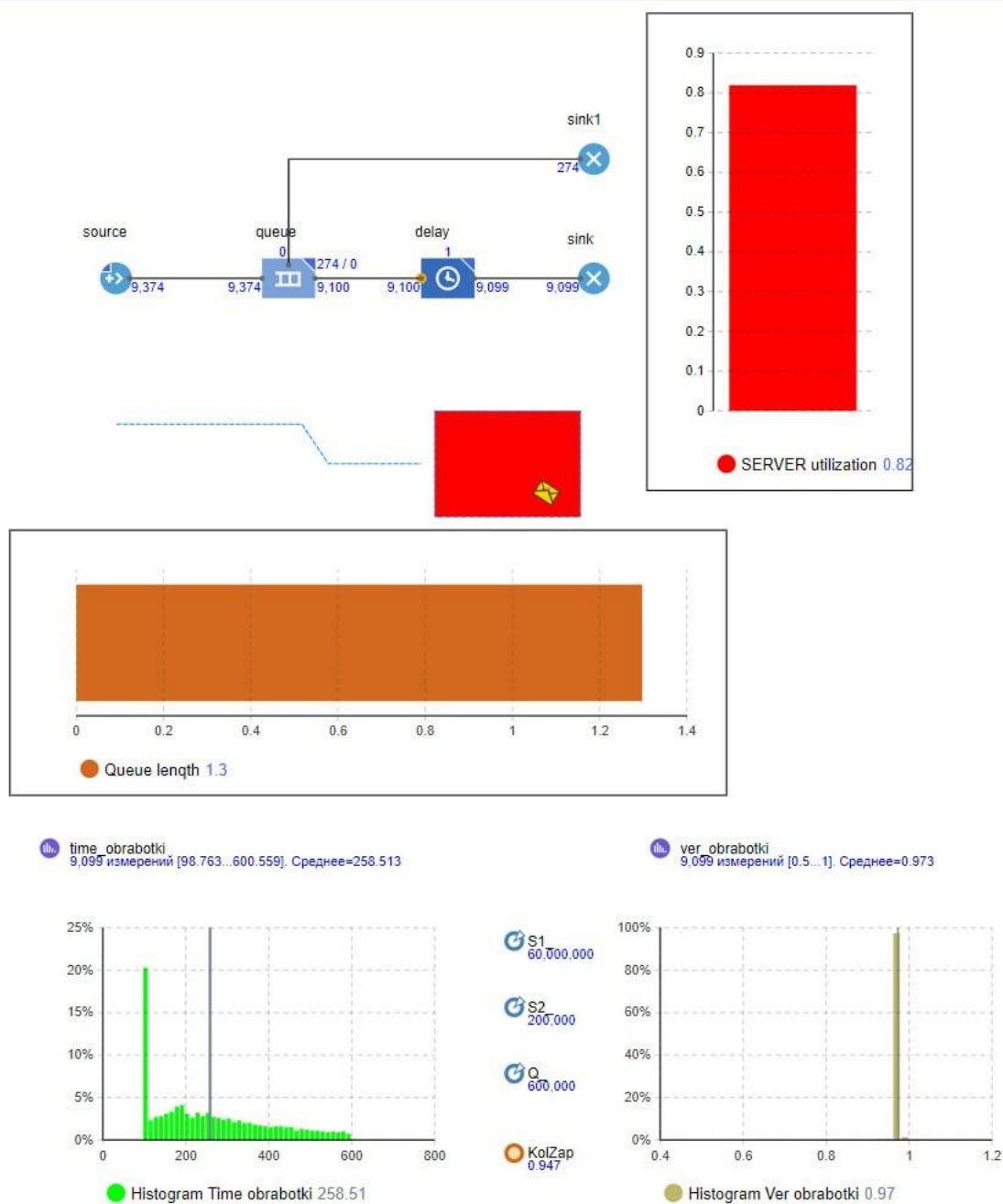


Рисунок 27 – Запуск модели

Часть 9. Интерпретация результатов моделирования

28. Для проведения исследований на модели сделайте ещё несколько дополнений и изменений. Можно было бы обойтись и без них, но они улучшат эксплуатацию модели. Из палитры Основная перетащите три элемента Параметр на диаграмму агента Main.

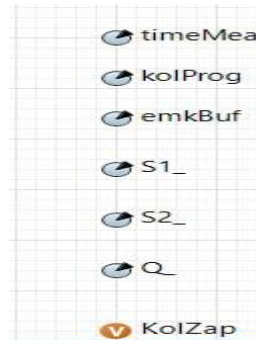


Рисунок 28 – Добавление трех параметров

29. Запустите модель.

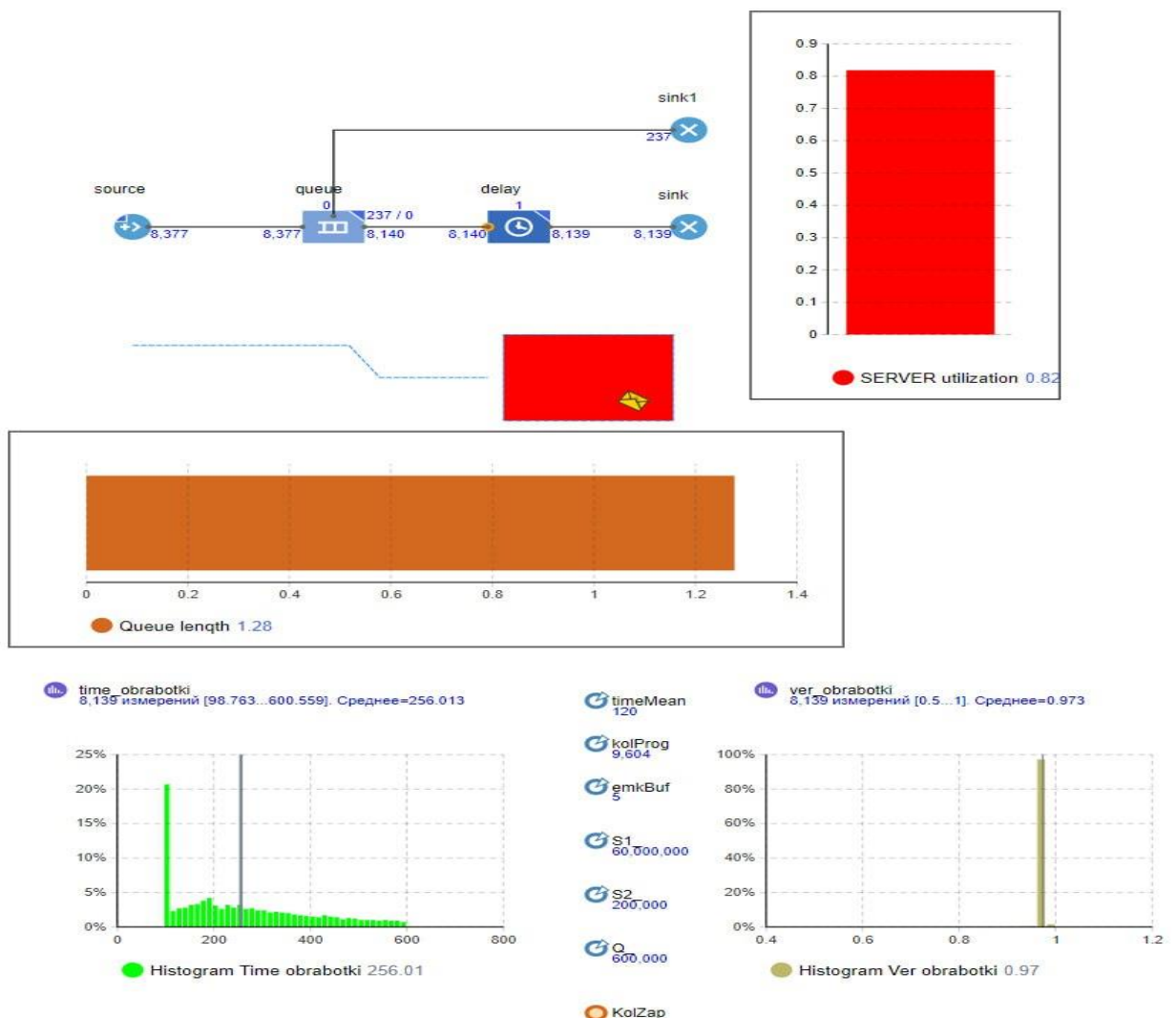


Рисунок 29 – Запуск модели

Вывод

В ходе данной практической работы мы построили модель обработки запросов сер.

Теперь мы лучше понимаем, как работает однофазная система массового обслуживания разомкнутого типа с ограниченной входной емкостью, то есть с отказами, и абсолютной надёжностью.

В процессе построения данной модели мы овладели базовыми знаниями о ресурсах AnyLogic и приемах работы с ними. Так же научились описывать процессы и собирать статистику для дальнейшей работы с ней.